

基于 Spark 的个性化音乐推荐系统设计与实现

赵吉^{1,2}, 胡海然¹, 张婷¹

(1. 无锡城市职业技术学院, 江苏无锡, 214000; 2. 大数据智能应用工程技术研究开发中心, 江苏无锡, 214000)

基金项目: 江苏省职业院校学生创新创业培育计划项目 (苏教职函〔2023〕36号): 基于Spark机器学习的智能推荐系统构建与实现、江苏省高等教育教改研究课题[023JSJG553]。

摘要: 本文设计了一个基于Spark的个性化音乐推荐系统。将爬取的音乐数据上传至Hadoop集群, 然后利用Spark机器学习交替最小二乘推荐算法对集群上的数据进行处理, 优化推荐模型超参数并计算AUC评分, 最终达到个性化推荐音乐的目的。

关键词: 推荐系统; Spark机器学习; 交替最小二乘推荐算法; AUC

中图分类号: TP311 **文献标志码:** A

0 引言

近年来, 数据的爆炸式增长和数据背后隐藏的巨大商业价值催生了一代代大数据处理技术, 而 Spark 是大数据领域中目前非常活跃的技术之一^[1]。随着人工智能的崛起, 未来无论是对于商业公司还是个人, 大数据的数据挖掘、数据分析以及基于大数据的人工智能应用都会成为提高竞争力的关键因素^[2]。因此, 本文基于 Spark 开发一款个性化音乐推荐系统, 该系统采用交替最小二乘法 (Alternating Least Squares, ALS) 模型计算 AUC 评分并依据评分由高到低输出推荐艺术家的名字, 以达到音乐推荐的目的。

1 系统设计

本系统发挥 Spark 在大数据处理方面的优势, 对特定用户推荐音乐艺术家。系统的重点和难点是调用 ALS 模型并合理设置参数, 计算 AUC 评分功能的实现。整个项目分为: 数据爬取、数据存储、数据清洗与数据分析四个流程设计, 如图 1 所示。

2 关键技术

2.1 数据采集

爬虫是通过编写程序模拟浏览器获取网络之上的相关的数据与信息, 被广泛用于互联网搜索引擎或其他类似网站, 可以自动采集所有其能够访问到的页面内容, 以获取或更新这些网站的内容和检索方式。爬虫一般分为数据采集, 处理, 储存三个部分。本系统采用 BeautifulSoup^[3] 实现网络爬虫数据采集。BeautifulSoup 是一个高效的网页解析库, 支持不同的解析器, 可以从 HTML、HTML5 和 XML 文件中提取数据。

2.2 数据存储

Hadoop 是 Apache 软件基金会的一款开源软件, 允许用户使用简单的编程模型实现跨机器集群对海量数据进行

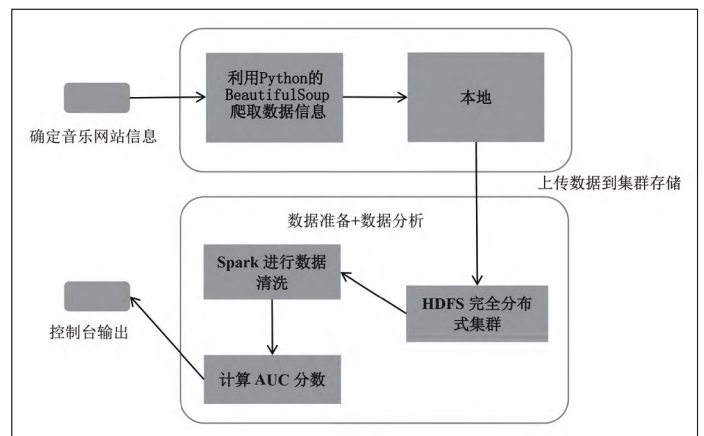


图1 项目整体流程图

分布式计算处理。本系统主要使用 Hadoop 的 HDFS 组件对数据进行存储^[4]。HDFS 是横跨在多台计算机上的存储系统, 主要解决大数据存储问题。它提供统一的访问接口, 访问分布式文件管理系统与访问普通文件系统一样, 拥有高容错的特点, 适应具有大数据集的应用程序。

2.3 数据分析

2.3.1 Spark 技术

Apache Spark 是一个开源框架, 作为计算引擎, 它把程序分发给集群中的许多机器, 同时提供一个编程模型。Spark 继承了 MapReduce 的线性扩展性和容错性, 同时对它做了一些重量级的扩展。它的 Dataset 和 DataFrame 抽象将流水处理线上的任何点物化在跨越集群节点的内存中。这样后续步骤如果需要相同数据集就不必重新计算或从磁盘加载。这使 Spark 可以应用于以前分布式处理引擎无法胜任的应用场景中。本系统采用 Spark 中的 Spark RDD、DataFrame 和 MLlib 技术进行大数据的离线批处理、机器学习分析解决推荐系统中最重要的任务和问题。

2.3.2 交替最小二乘推荐算法 (ALS)

交替最小二乘法 (Alternating Least Squares, ALS)^[5] 被收录于 Spark 的 Mlib 以及 ML 库中, 其亮点之一就在于优化参数时使用了交替最小二乘法, 而非梯度下降算法, 使

得 ALS 算法可以进行分布式并行计算。

ALS 推荐模型是一种基于协同过滤的推荐算法，它采用了矩阵分解的技术来预测用户对物品的偏好。ALS 模型由用户特征矩阵和物品特征矩阵组成，它们都是低秩矩阵，通过将低秩矩阵分解为两个较低秩的矩阵，可以准确地预测用户对物品的偏好。ALS 模型的优势在于它可以在不知道物品相似性的情况下推荐准确的物品，并且可以根据用户的行为快速更新推荐结果。

3 系统实现

■ 3.1 数据集

本系统采用的数据使用的是 Audioscrobbler 爬取的数据集。该数据集只记录了播放数据，播放记录包含的信息要比评分数据少，比如“Jack 播放了一首 Jaekwon 的歌曲”，虽然 Jack 播放了这首歌，但不代表他喜欢。人们虽然经常欣赏音乐，但是很少给出评分，所以该数据集要大很多，包含了更多的用户和艺术家，通过用户和艺术家之间的隐式数据反馈，可以看出用户和艺术家之间的关系。数据集由 3 个 txt 文件组成。artist_data.txt 文件包含的是（歌曲 ID，歌曲名字）。user_artist_data.txt 中包含了 14 万个用户 ID 和 160 万个艺术家姓名，记录了两千多万条用户播放的信息，是主要数据集。

播放记录在记录时，是有可能出现错误数据的，客户端提交的艺术家的名字如果有拼写错误，或者没有正确使用标准符号时，是不会立刻被发现的。因此，为了将这些错误的艺术家 ID 对应到正确的艺术家 ID，artist_alias.txt 文件包含了艺术家的名字以及对应的 ID。

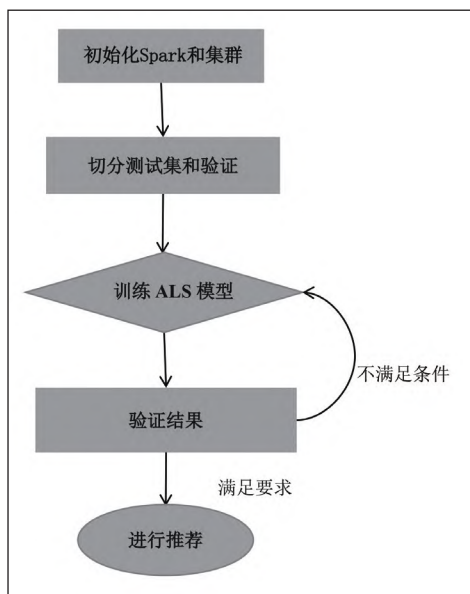


图2 ALS 推荐算法模型流程图

■ 3.2 基于 ALS 推荐算法的 Spark 数据分析

将爬取的 3 个 txt 文件上传至 Hadoop 完全分布集群的 master 节点后，采用基于 ALS 推荐算法对数据进行训练并给出推荐结果，如图 2 所示。

3.2.1 数据准备

在 Spark MLlib 中实现 ALS 算法的推荐模型使用 Int 类型时效率较高，但是 ID 的数值必须在最大取值 (Int.MaxValue) 范围内，即 $2^{31}-1$ ，所以先验证数据集。使用 textFile 方法读取 txt 类型的数据集，将数据文件的类型转换成 String 类型。数据集的每行分别是：用户 ID，艺术家 ID 和被播放次数，每列中间使用空格进行分隔，所以采用空格拆分每行，将用户 ID 和艺术家 ID 的数据解析为整数类型，即数据类型为 Int 类型。利用 agg 函数算出用户 ID 和艺术家 ID 两列的最大值和最小值，均在 Int 类型的取值范围内，所以在解析时没有必要进行转换。artist_data.txt 存储的是艺术家的名字以及难以分辨的数字艺术家 ID。使用 span() 函数解析这个数据集时，每行用第一个制表符分割，第一部分为艺术家 ID，第二部分为艺术家名字。为了避免数据集中非法数据产生 NumberFormatException 错误，使用 flatMap() 函数，将每个对应的集合展开后放入一个更大的集合，返回 DataFrame，其中“ID”和“name”对应的是艺术家 ID 和名字。artist_alias.txt 数据集中每行有一个错误 ID 和一个正确 ID，用“\t”分隔，先把两个 ID 转为 Map 集合，然后将错误的艺术家 ID 映射到正确的艺术家 ID。在解析数据集过程中同时利用 Spark 的 filter 函数，去除数据集中的重复数据，删除缺失值记录等清理操作，确保构建模型时数据集的完整性。

3.2.2 构建模型

设置辅助函数将播放记录中错误的艺术家 ID 根据 artist_alias 数据集转换成正规 ID，同时将输入的行转换成合适的列。为了节省内存和网络流量，创建一个广播变量使 Spark 对集群中的每一个执行器 (executor) 只发送一个副本。因为 ALS 算法是迭代形式的，使用 cache() 函数可以持久化数据，避免因使用 DataFrame 时从原始数据开始重新计算而浪费计算内存。

构建模型时，系统共有超过 170 万个特征向量。该模型采用两个 DataFrame 表示两个大型矩阵，分别是“用户 - 特征”以及“产品 - 特征”。因为初始特征向量是随机选择的，所以每次运行的结果可能会不同导致最终模型会有不同。对于 ALS 算法，预测是一个模糊值，值域是 0~1。“prediction”的值越大，说明推荐引擎更加优秀，推荐的结果更加符合预期的结果，也可以将其看成 0/1 值的估算，0 表示用户不喜

欢的艺术家歌曲，1 则表示用户喜欢的艺术家歌曲。

3.3 个性化推荐系统设计

3.3.1 计算 AUC

为了使推荐引擎发挥推荐用户没有听过的艺术家功能，从原数据集中取出一部分艺术家的播放数据，作为指标来判断引擎优劣。被取出数据中的艺术家可以推荐给每一位用户，但是这些数据不让推荐引擎参与分析。推荐引擎对模型中所有产品评分，并对比检查其余的艺术家的推荐排名情况。

通过比较已经被取出的音乐家的推荐排名和整个数据集的艺术家推荐排名，可以得出推荐引擎的得分。观察集合中已经被取出艺术家排名高的组合所占的比例就是推荐模型的得分。最好为 1.0，随机给模型的期望得分为 0.5，0.0 代表最差。

这个指标和接收者操作特征曲线（Receiver Operating Characteristic, ROC）相关，上段中的指标等于 ROC 曲线下方的面积，称为 AUC（Area Under the Curve），即随机抽取一个阳性样本和一个阴性样本，引擎正确判断正样本的值高于负样本的概率就是 AUC 评分，如式 (1) 所示。它反映出优秀的随机选择推荐比糟糕的随机选择推荐的排名高的概率。推荐系统中，每个用户计算 AUC 并取平均值，所得出的结果指标会与之前单独用户的 AUC 评分不同，称为“平均 AUC”。

$$AUC = \frac{\sum_{ins_i \in positiveclass} rank_{ins_i} - \frac{M \times (M + 1)}{2}}{M \times N}$$
 式 (1)

将数据集分成两个子集：训练集和交叉验证集（Cross-Validation, CV）。具体实现接受一个交叉验证集和一个预测函数，交叉验证集表示每个用户的正样本艺术及。预测函数把“预测”加入每一个有“用户 - 艺术家”的 DataFrame，形成新的 DataFrame，这里的“预测”表示用户和艺术家的关联度，“预测”值越高，推荐的排名也越高。为了训练 ALS 模型，将 90% 的原始数据取出作为训练集，还有 10% 的原始数据作为验证集，用于评估模型的优劣。结果约为 0.876，这说明使用 AUC 指标，推荐引擎在非个性化推荐方面已经提升。

3.3.2 选择超参数

想要更具有“个性化”的推荐，需要对表 1 所示模型超参数进行设置。设置模型的超参数 Alpha、rank 以及 RegParam 不会表现在推荐模型的内部矩阵中，因为这些矩阵只是参数，数值的大小是由算法决定的。本系统中尝试了 8 种组合：rank 为 5 或 30，RegParam 为 4.0 或 0.0001，alpha=1.0 或 40.0。组合的结果按照 AUC 数值的由高到低

排序输出，如图 3 所示。从图 3 可以看出虽然这些值的相差很小，但是对于 AUC 评分而言，还是可以得出高低的。从结果可以看出 alpha 取 40 的时候 AUC 值要比取 1 时更高。说明模型在强调用户听过什么歌曲时比强调用户没听过什么歌曲时的表现更好。regParam 取值较大的时候看起来结果更加优秀，说明模型可能需要较大的过拟合参数值来减少过拟合的影响，防止过度精确拟合用户的稀疏输入数据。

表1 超参数及描述

参数名	参数描述
rank	ALS模型中设置潜在因素的个数，“用户-特征”和“产品-特征”矩阵中，拆分集合的k值(特征列数量)。
MaxIter	最大迭代次数，如果迭代次数越多的话，矩阵分解结果越好，坏处是花费的时间也会增多。
RegParam	正则项的参数，过拟合参数(lambda)，控制损失函数与正负项的比例，防止整个训练过程过拟合，默认为0数值。越大越不容易产生过拟合。
Alpha	将权值的增量乘以系数 a，防止它过大发散。控制矩阵分解时，已经被发现的“用户-产品”交互相较于没被发现的交互的权重。

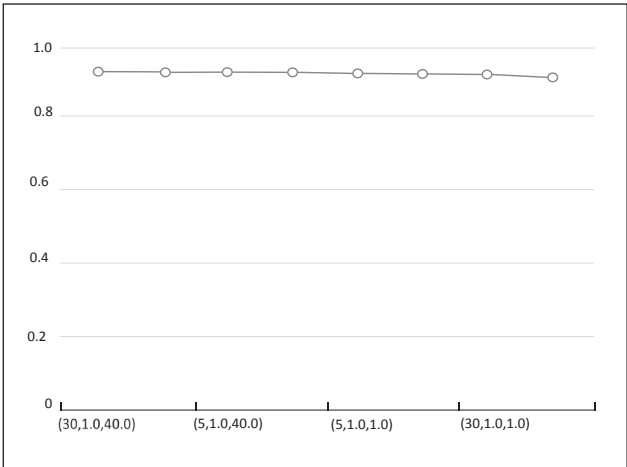


图 3 设置超参数后 AUC 评分对比折线图

3.3.3 产生推荐

为了检验推荐模型给出的艺术家推荐名单是否合理，随机挑选 ID 为“2093760”的用户，获取该用户播放过的艺术家 ID，如图 4 所示。

从结果可以看出该用户播放的艺术家有嘻哈风格、流行音乐风格以及摇滚风格。对该 ID 的用户使用推荐，结果对比如图 5 所示。

```
+-----+-----+
| id | name |
+-----+-----+
| 1180 | David Gray |
| 378 | Blackalicious |
| 813 | Jurassic 5 |
| 1255340 | The Saw Doctors |
| 942 | Xzibit |
+-----+-----+
```

图 4 查看特定 ID 用户播放过的艺术家姓名运行结果

图 5(a) 表示未优化前结果显示，模型推荐的所有音乐家都是嘻哈风格，与实际情况相差甚远，虽然能看出模型推荐的艺术家是大多数人喜爱的，但是没有针对用户的个性化。使用最优超参数加入新的推荐模型当中得到的结果如图

5(b) 所示，从结果可以看出这次的推荐更符合这位用户的个性化，与之前全部是嘻哈风格的推荐不同，这次的推荐其中大部分是流行或者摇滚风格。这里的 [unknown] 并不是某位艺术家的名字，查看原始数据集，发现 [unknown] 出现了约 42 万次。[unknown] 是没有艺术家姓名信息的时候，[unknown] 将会作为默认值被播放记录返回。



图 5 推荐结果

4 结束语

本文提出了基于 Spark 技术采用 ALS 模型计算 AUC 评分并依据评分由高到低输出推荐艺术家的名字的音乐推荐系统的设计与实现。通过测试结果对比了参数优化后对个性化推荐的影响，证明本系统达到了个性音乐推荐的目的。本

系统对模型的参数设置还可以进一步调试优化，排查输入数据中的异常情况，推荐结果会更加令人满意。在实际使用中，推荐引擎需要实时给出推荐，从而导致推荐引擎需要在客户浏览视频页面时频繁给出推荐。在大规模数据量下，系统可以预先计算得到推荐结果存储到数据库中，但是缺点就是需要对所有可能提供快速推荐的用户进行预计算，无法预测是哪一个用户需要预计算，这样会产生很多无用的运算。这也是本系统下一步需要进行的研究工作。

参考文献

* [1] 肖芳, 张良均. Spark 大数据技术与应用 [M]. 北京邮电出版社, 2018:240-251.

* [2] 谷广兵, 顾佩佩. 基于大数据平台的推荐系统研究与实现 [J]. 现代信息科技, 2023,7(01):26-29.

* [3] Ryan Mitchell, 陶俊杰, 陈小莉译. Python 网络数据采集 [M]. 人民邮电出版社, 2016:25-50.

* [4] 董新华, 李瑞轩, 周湾湾, 等. Hadoop 系统性能优化与功能增强综述 [J]. 计算机研究与发展, 2013,50(S2):1-15.

* [5] 李耀飞. 基于 Spark 的协同过滤推荐系统的研究 [D]. 东华大学, 2018.

作者简介: 赵吉 (1980-), 女, 江苏泰兴人, 博士, 教授, 研究方向为群体智能算法、大数据应用开发技术。

(上接第 50 页)

重要方向。而移动灌溉喷头作为现代农业中不可或缺的技术装备，通过其高效的水资源利用和精确的灌溉方式，显著提升了农田的产量和质量。我们深入分析了现代农业遇到的问题，改进现有技术，利用移动灌溉喷头的功能设计、硬件设计、软件设计，通过传感器将相关数据上传至云端，由云端判断或用户 APP 控制，上传指令到执行器实现相关功能。移动灌溉喷头在不同作物种植中展现了多样化的应用模式，满足了不同农作物对水分需求的差异性，为农业生产带来了更加稳定和可靠的灌溉保障。我们将进一步深入研究其不同地区和环境条件下的适用性及优化空间，以推动其技术革新和应用推广，为全球农业生产提供更为可持续和高效的解决方案。

参考文献

* [1] 陈启和, 陶玲, 唐艳. 蔬菜大棚智能监控系统设计 [J]. 自动化应用, 2024,65(07):191-193.

* [2] 董邵万, 郭锐. 机器人驱动模块 L298N[J]. 数码世界, 2018(07):79-80.

* [3] 郑晓庆, 杨日杰, 杨立永, 等. 多路输出 DC-DC 电路设计 [J]. 国外电子测量技术, 2012,31(09):31-33.

* [4] 刘火良, 杨森编著. STM32 库开发实战指南: 基于 STM-32F4[M]. 北京: 机械工业出版社, 2017.02.

* [5] 乔凌霄, 郭超维, 刘源涛, 等. 基于超声传感器的避障小车系统设计 [J]. 运城学院学报, 2019,37(03):12-15.

* [6] 吕嫫. 基于 ESP32-CAM 的无线监控小车系统设计 [J]. 科技创新, 2023(24):1-4.

* [7] 卜永波, 罗小玲, 陈一. 基于 DHT11 传感器的温湿度采集系统 [J]. 计算机与现代化, 2013(11):133-135.

* [8] 张文彪, 何亮, 张俊辉, 等. 基于传感和互联技术的智慧乡村综合信息监管系统研发 [J]. 电子制作, 2024,32(06):99102.

作者简介: 黄天翔 (2003-), 男, 汉族, 广东湛江人, 本科, 研究方向为物联网工程。